

Module 19 Week 22 Required Assignment

1. Trade-offs between model size, inference speed, and accuracy in LLMs

Larger models generally produce higher-quality, more nuanced outputs (better reasoning, fewer factual errors, better handling of ambiguous instructions), but they cost more per token to run, need more memory/GPU capacity, and respond more slowly. Smaller models are cheaper and much faster to run - often enabling on-device or low-latency deployment - but they can be less accurate, more prone to hallucination on complex requests, and less robust on tasks outside their training distribution.

For a real-time chatbot, latency is usually the dominant constraint - users expect a response within roughly a second. This pushes the choice toward a smaller or distilled model (or a larger model that's been quantised) that responds quickly enough, accepting some accuracy loss on the hardest queries. This is often supplemented by routing only the genuinely hard/ambiguous queries to a larger, slower model in the background ("model cascading"), so most simple requests stay fast while difficult ones still get high-quality handling.

2. Combining multiple LLMs for a complex application

Scenario: a customer support platform that needs to (a) classify incoming tickets by urgency/category, (b) draft a suggested reply, and (c) flag anything referencing legal or safety issues for human escalation.

Using a single large general-purpose model for all three steps is wasteful and slower than necessary. Instead:

- A small, fast classifier model handles ticket categorisation and urgency scoring on every incoming ticket (cheap, needs to run at high volume).
- A larger, higher-quality generative model drafts the actual suggested reply only for tickets that pass classification (lower volume, needs better language quality).
- A specialised safety/compliance model (or a rules+LLM hybrid) scans for legal/safety flags, since that needs high recall on a narrow, well-defined task rather than general capability.

Integration approach: treat each model as a service behind its own API endpoint, with a lightweight orchestration layer (a backend service or workflow engine) that calls them in sequence, passes the relevant output of one as input to the next, and applies business logic (e.g. "if flagged, skip auto-reply and escalate") between calls. This keeps each model doing what it's individually best/cheapest at, rather than forcing one large model to do everything.

3. AI-powered autonomous agents for software development

Advantages:

- Can handle repetitive, well-specified tasks end-to-end (writing boilerplate, running and fixing failing tests, upgrading a dependency across many files) faster than a human doing it manually.
- Can work through long, multi-step tasks in the background (e.g. "add this feature, write tests, open a PR") while a developer focuses on higher-judgement work.
- Lowers the barrier for exploring an unfamiliar codebase, since the agent can search, read and summarise code much faster than a human onboarding.

Challenges:

- Agents can go down the wrong path on ambiguous or under-specified tasks, producing plausible-looking but incorrect or overengineered changes if not checked.
- They can lack full context on business priorities, security implications or team conventions unless those are made explicit, so unsupervised changes carry real risk (e.g. an agent "fixing" a failing test by weakening the assertion instead of fixing the underlying bug).
- Debugging an agent's mistake can sometimes take longer than doing the task manually, especially for subtle logic errors buried in a large generated diff.

Example: an agent asked to "add input validation to all API routes" might correctly add checks to routes with obvious required fields, but miss a route with more complex conditional validation logic, requiring a human to review and patch the gaps - illustrating why human review of agent output remains necessary today.

4. Impact of autonomous AI agents on the role of human developers

Short-term: the developer's role shifts from writing every line of code toward specifying, reviewing and directing - writing clear task descriptions, reviewing agent-generated diffs, and course-correcting when an agent misunderstands intent. Routine, well-defined work (boilerplate, test-writing, simple bug fixes, dependency upgrades) increasingly gets delegated to agents, freeing developer time for architecture decisions, ambiguous problems, and cross-team coordination that still require human judgement.

Long-term: as agents get more reliable at longer, more ambiguous tasks, the value of a developer increasingly concentrates in areas agents are weak at - understanding **why** something should be built a certain way (product judgement, security/compliance trade-offs, system design at scale) rather than **how** to type it out. This doesn't eliminate the need for developers, but it does mean the skill that's valued shifts from raw code production toward specification, review, and judgement about what to build and why - skills that are harder to fully automate.